The Architecture of Latency: A Technical Roadmap for the LocalPlus Sub-300ms MVP Build

The transition from a search-based internet to an answer-based economy necessitates a fundamental reimagining of local discovery infrastructure. The traditional model of "Keyword-Link-Click" is being rapidly superseded by the "Intent-Answer-Action" paradigm, where the value lies not in directing a user to a webpage, but in providing an immediate, grounded, and actionable response. LocalPlus represents a post-search discovery layer designed to solve the "Hyper-Local Decay" problem—the tendency of centralized large language models (LLMs) to rely on stale, city-level data regarding merchant availability and civic hierarchies.[1] To achieve a sub-300ms response time for users in logically disparate regions like Sydney, Australia, and Bangkok, Thailand, the infrastructure must leverage edge-native computing and high-velocity retrieval mechanisms. This report outlines the technical requirements for building the LocalPlus MVP for $20 to $150 per month, utilizing the Bunny.net ecosystem and Google Gemini 1.5 Flash.

Anycast Orchestration: Deploying the Model Context Protocol at the Edge

The Model Context Protocol (MCP) serves as the universal orchestration bridge, acting as a standardized interface between the generative intelligence of the LLM and the structured reality of local merchant data.[1] To meet the stringent latency budget of 300ms, the MCP server cannot reside in a single centralized data center; it must be distributed globally via an Anycast network.

The Mechanism of Anycast Routing in Magic Containers

Unlike traditional Unicast routing, where a single IP address corresponds to a single physical server, or GeoIP-based routing, which relies on potentially inaccurate DNS mapping, Anycast allows multiple Bunny.net nodes to share a single IP address.[2] The Border Gateway Protocol (BGP) ensures that network traffic is routed to the "closest" node based on network hops rather than pure geographical distance.[2]

For the LocalPlus MVP, Bunny.net Magic Containers provide the ideal environment for this deployment. The platform utilizes bare-metal servers and NVMe storage across 40+ regions to ensure that the compute layer is physically adjacent to the user.[4] When a user in Bangkok initiates a request, the BGP routes the packet to the Bangkok Point of Presence (PoP), while a user in Sydney is served by the Sydney PoP, maintaining a median Time to First Byte (TTFB) as

low as 26ms.[1]

Step-by-Step Deployment of the MCP Server

The deployment of an MCP server on Magic Containers requires a shift from standard web hosting to containerized edge execution. The following technical steps define the workflow for establishing a persistent, Anycast-routed gateway.

Deployment Phase	Action Item	Technical Requirement
Orchestration	App Creation	Select "Advanced Deployment" in the Bunny.net Dashboard.[5]
Regionality	Base Region Setting	Designate Sydney (SYD) and Bangkok (BKK) as "Base Regions".[5]
Compute	Image Selection	Utilize a Rust or Python-based Docker image for the MCP server.[1]
Networking	Endpoint Config	Add an "Anycast" endpoint mapping internal port 8080 to public port 443.[7]
Scaling	Instance Limits	Set a minimum of 1 instance per region to eliminate cold-start latency.[5]

The choice of "Base Regions" is critical for the LocalPlus MVP. By designating Sydney and Bangkok as base regions, the system disables dynamic provisioning for those specific areas, ensuring the application is always "warm" and ready to respond without the 60-second delay sometimes associated with SafeHop rerouting during node health monitoring.[1]

Containerization and Environment Configuration

The MCP server must be lightweight to ensure rapid startup and minimal memory overhead. Using a distroless or slim Python image ensures that the container is highly portable and secure.[6]

Dockerfile

```dockerfile
FROM python:3.11-slim
WORKDIR /app
# Set environment variables for high-performance Python
ENV UV_SYSTEM_PYTHON=1 \
  PYTHONUNBUFFERED=1 \
  PORT=8080
# Install dependencies including FastMCP for protocol handling
COPY requirements.txt.
RUN pip install --no-cache-dir -r requirements.txt
COPY server.py.
EXPOSE 8080
# Run uvicorn with proxy-aware headers for Anycast compatibility
CMD ["uvicorn", "server:app", "--host", "0.0.0.0", "--port", "8080", "--proxy-headers"]
```

The server logic utilizes the FastMCP framework to create a bridge between the LLM and the local merchant database. In a production environment for 2026, this implementation often leverages Server-Sent Events (SSE) to maintain a hanging HTTP connection, allowing for real-time streaming of context updates.[1]

High-Velocity Vector Retrieval: Engineering the libSQL Search Layer

The "Moat" of LocalPlus is its ability to handle complex relational queries—such as finding a premium-status merchant with real-time availability near a specific civic landmark—using a hybrid of vector search and graph traversals.[1] For the MVP scale of 500 local businesses, the Bunny Database (built on libSQL) provides a globally distributed, SQLite-compatible solution that minimizes retrieval latency.[13]

The libSQL Vector Extension and DiskANN Algorithm

The 2026 Bunny Database integrates the DiskANN algorithm to facilitate Approximate Nearest Neighbor (ANN) search. DiskANN is specifically designed to handle high-dimensional vectors at scale by navigating through multiple layers of coarse-to-fine approximations.[15] This handles the requirement of querying 500 business records in under 50ms by avoiding brute-force linear scans.

The similarity between the user's intent and the merchant's profile is calculated using the cosine distance, expressed as:

$$d_{\cos}(u, v) = 1 - \frac{u \cdot v}{\|u\|\|v\|}$$

LibSQL stores these embeddings in a specialized F32_BLOB column, which provides single-precision (32-bit) floating-point representation.[15]

Implementation of the Vector Schema and Search Syntax

To achieve sub-50ms performance, the database must be correctly indexed and queried using the vector_top_k table-valued function. This bypasses the overhead of standard SQL ORDER BY clauses when dealing with high-dimensional data.[16]

SQL

```sql
-- Step 1: Initialize the merchant table with a vector column
CREATE TABLE merchants (
  id INTEGER PRIMARY KEY,
  name TEXT,
  category TEXT,
  status TEXT,
  embedding F32_BLOB(768) -- Dimension matches Gemini 1.5 Flash
);

-- Step 2: Create the vector index using the DiskANN engine
-- Parameters fine-tune the balance between recall and speed
CREATE INDEX merchants_idx ON merchants (
  libsql_vector_idx(embedding, 'metric=cosine', 'compress_neighbors=float8')
);
```

For querying, the syntax must explicitly utilize the index to maintain performance. A common pitfall in libSQL is the inversion of join orders when WHERE predicates are applied to a regular table joined with the vector index, which can increase latency from 10ms to several seconds.[21] The use of CROSS JOIN or a subquery is required to force the query planner into the correct execution path.[21]

SQL

```sql
-- Step 3: High-performance vector retrieval (Sub-50ms)
SELECT m.name, m.status, v.distance
FROM vector_top_k('merchants_idx', vector32('[0.12, -0.05,...]'), 10) AS v
CROSS JOIN merchants AS m ON m.rowid = v.id
WHERE m.status = 'Available';
```

Performance Optimization and Data Compression

For an MVP with 500 records, the use of FLOAT32 vectors is recommended for accuracy. However, as the system scales to millions of vectors, libSQL supports more compact storage classes like FLOAT8 or even FLOAT1BIT.[15]

Vector Type	Storage Requirement	Use Case Suitability
FLOAT64	$8D +$ bytes	High-precision scientific reasoning.
FLOAT32	$4D$ bytes	**Recommended** for standard RAG and local discovery.[15]
FLOAT8	$D +$ bytes	Scaling to 10M+ records with minimal precision loss.[15]
FLOAT1BIT	$\lceil D/8 \rceil +$ bytes	Ultra-compact edge deployments with simple ranking.[15]

By using FLOAT32, the LocalPlus MVP ensures a high "Recall@K"—meaning the proportion of true nearest neighbors found in the top results—without exceeding the sub-50ms retrieval window.[22]

Generative Optimization: Achieving Sub-200ms TTFT with Gemini 1.5 Flash

The generative layer of the LocalPlus MVP utilizes Gemini 1.5 Flash, a model specifically built for speed, efficiency, and massive context windows.[24] To meet the sub-300ms goal, the infrastructure must minimize Time to First Token (TTFT) by leveraging explicit context caching

and highly constrained prompting strategies.[26]

The Flash Optimization Strategy

The TTFT for Gemini 1.5 Flash is influenced by token count, model complexity, and network conditions.[29] For the LocalPlus MVP, the goal is to shave 150-200ms off the standard response time.

Explicit Context Caching

Context caching is the primary lever for reducing latency. By caching the static system instructions and common merchant metadata (the "Grounding Pack"), the model avoids re-processing these tokens for every user request.[27] This is particularly effective when the static prefix exceeds 1024 tokens.[30]

System Prompt and Response Schemas

Instruction following is significantly improved in Gemini models when using XML-tagged instructions rather than standard prose. Furthermore, forcing the model to use a response_schema with enums eliminates "preamble chatter" (e.g., "Sure, I can help you with that"), which directly reduces TTFT by cutting unnecessary output tokens.[28]

Recommended System Prompt for LocalPlus:

XML

```
<system_instruction>
  <role>LocalPlus Discovery Agent for Sydney/Bangkok.</role>
  <objective>Answer user intent using the provided merchant context.</objective>
  <output_spec>
    - Format: Concise prose with symbol injection.
    - No_Preamble: true.
    - Max_Length: 50 tokens.
  </output_spec>
</system_instruction>
```

Configuring the Thinking Budget

Recent iterations of the Flash series (e.g., 2.0 and 2.5) introduce dynamic reasoning or "thinking" tokens. While this enhances intelligence for complex tasks, it adds significant latency. For the LocalPlus discovery task, setting the thinkingBudget to 0 or using the "minimal" level is essential

to ensure that the model begins generating the answer immediately.[30]

Zero-Latency Context Passing

Context passing between the Bunny MCP server and Gemini is achieved by using the cached_content ID in the request configuration. This ensures that the context resides in the model's "short-term memory" (the context window) rather than being transmitted over the network for every turn of the conversation.[27]

Configuration Parameter	Setting for MVP	Rationale
Model	gemini-1.5-flash	Optimized for low-latency, high-throughput.[24]
Streaming	stream: true	Improves the perception of latency to the human user.[34]
Temperature	0.0	Ensures deterministic, consistent business rankings.[34]
Thinking Budget	0	Bypasses reasoning monologues to reduce TTFT.[28]
Safety Settings	BLOCK_NONE	Minimizes latency by reducing safety-filter overhead.[35]

The Symbolic Injection Layer: Real-Time Stream Transformation

LocalPlus monetizes discovery via a "Language of Visibility," where commercial influence is signaled through UTF-8 symbols (·, †, ★) embedded in the text.[1] To maintain the sub-300ms budget, these symbols must be injected into the LLM token stream in real-time without buffering the entire response.

Deno Edge Scripting and TransformStreams

Bunny.net Edge Scripting, built on Deno, provides access to the Web Streams API. The TransformStream interface allows the script to modify text chunks packet by packet as they are received from the Gemini API.[36] This pattern is essential for constant-memory processing and

minimal latency.

The injection logic utilizes a high-performance string matching algorithm (such as Aho-Corasick) to identify business names in the token stream and append the appropriate symbol based on a lookup in the Bunny Edge Cache (Perma-Cache).[1]

Deno-Based Symbolic Injection Code

The following script acts as a middleware layer, intercepting the stream from Gemini and augmenting it with monetization symbols.

TypeScript

```typescript
import * as BunnySDK from "https://esm.sh/@bunny.net/edgescript-sdk@0.11.2";

/**
 * Symbolic Injection Middleware
 * Appends symbols (·, †, ★) to merchant names based on edge cache metadata.
 */
async function onOriginResponse(context: { request: Request, response: Response }):
Promise<Response> {
  const symbolMap: Record<string, string> = {
    "PROMOTED": "·",
    "PRIORITY": "†",
    "PREMIUM": "★"
  };

  const transformStream = new TransformStream({
    async transform(chunk, controller) {
      const decoder = new TextDecoder();
      const encoder = new TextEncoder();
      let text = decoder.decode(chunk);

      // Regex to find potential merchant names (e.g., within brackets)
      const regex = /\[Merchant: (.*?)\]/g;
      let match;

      while ((match = regex.exec(text))!== null) {
        const merchantId = match;
        // sub-5ms lookup in Bunny's localized edge cache
        const status = await BunnySDK.cache.get(`merchant_status_${merchantId}`);
```

```
      const symbol = symbolMap[status] |

| "";
      text = text.replace(match, `${merchantId}${symbol}`);
    }

    controller.enqueue(encoder.encode(text));
  }
});

const modifiedBody = context.response.body?.pipeThrough(transformStream);

return new Response(modifiedBody, {
  status: context.response.status,
  headers: context.response.headers
});
}

BunnySDK.net.http.servePullZone().onOriginResponse(onOriginResponse);
```

By using the pipeThrough method, the stream remains open, allowing the user to see tokens as they are generated. This prevents "backpressure" from building up, which would occur if the entire response were loaded into memory for modification.[36]

Real-Time Truth: The Partner App "Direct Push" Ecosystem

A significant technical hurdle in local discovery is data freshness. Standard LLMs suffer from a knowledge cutoff, while even RAG systems can have an indexing lag. LocalPlus solves this through a "Hot-Swap" mechanism, allowing merchant partners to update their availability status in real-time via the Hrana protocol over HTTP.[1]

The Hrana Protocol for Instant Updates

Bunny Database provides a SQL API that uses the libSQL remote protocol (Hrana). This enables simple POST requests to execute SQL statements directly against the globally replicated database without the need for complex SDKs or persistent connections.[39]

Design of the "Direct Push" Command

The Partner App utilizes a simple CURL or fetch command to update the status column in the libSQL database. Once the update is committed, Bunny Database automatically replicates the

change across all active regions.[13]

Bash

```
# Hot-swap merchant status from 'Available' to 'Sold Out'
curl -X POST https://[your-database-id].lite.bunnydb.net/v2/pipeline \
 -H "Authorization: Bearer [your-access-token]" \
 -H "Content-Type: application/json" \
 -d '{
  "requests":
      }
    },
    { "type": "close" }
  ]
 }'
```

The implications of this real-time update are profound. A merchant in Bangkok can update their availability, and a user query seconds later will reflect this change. The global "Truth" propagation time is approximately 458ms, ensuring that the AI never generates an answer that leads to a failed transaction (e.g., recommending a closed shop or a sold-out item).[1]

Economic Modeling: Scaling an Answer Engine for $150/Month

The LocalPlus MVP is designed to operate within a monthly budget of $20 to $150. This is achieved by shifting from the "Cost Per Click" model to a "Cost Per Answer Share" model, leveraging Bunny.net's no-request-fee pricing and Gemini's low-cost Flash tier.[1]

Monthly Infrastructure Budget (MVP Scale: 500 Merchants)

The following table models the expected costs for an MVP serving 1,000 requests per day in Australia and Thailand.

Infrastructure Component	Pricing Unit	Estimated Usage	Monthly Cost
Anycast IP Fee	Flat Fee	1 IP Address	$2.00 [3]

Magic Containers	$0.02 / 3,600 CPU sec	200,000 CPU seconds	$1.11 [3]
Memory (64MB chunks)	$0.005 / GB-hour	2GB / Hour (Avg)	$7.20 [3]
NVMe Storage	$0.10 / GB-month	10 GB	$1.00 [3]
Bunny Database Reads	$0.30 / 1B rows	500k rows scanned	< $0.01 [40]
Gemini 1.5 Flash API	$0.10 / 1M input tokens	100M tokens	$10.00 [24]
Bunny Edge Scripting	$0.20 / 1M requests	30k requests	$0.01 [41]
Bandwidth (Global)	$0.01 / GB	100 GB	$1.00 [3]
Total Estimated Cost			**$22.33 / Month**

This configuration leaves a significant surplus for scaling. Even at ten times the traffic, the monthly cost would remain under $100, demonstrating the superior unit economics of the Bunny.net/Gemini stack compared to traditional hyperscale cloud providers.

Transitioning to "Cost Per Answer Share"

In the generative era, SMEs will compete for "Answer Inclusion"—the frequency with which they are mentioned in AI-generated responses. LocalPlus monetizes this through four layers: Mentions (Awareness), Citations (Authority), Clicks (Intent), and Outcomes (Transaction).

The Symbolic Injection system provides the necessary telemetry to track these events. By tracking how many times the TransformStream injected a dagger symbol (†) versus a star symbol (★), the platform can provide merchants with real-time "Answer Share" metrics, justifying the subscription or pay-per-mention model.

Latency Synthesis: The 300ms Budget Breakdown

To provide a sub-300ms experience in Sydney and Bangkok, every millisecond must be

accounted for across the network, compute, and generative layers.

Logical Latency Budget (End-to-End)

Segment	Estimated Latency (ms)	Mechanism for Achievement
Network (Sydney/BKK)	17 - 24 ms	BGP Anycast routing to local PoP.[1]
MCP Middleware	10 - 20 ms	High-concurrency Rust/Go on bare-metal.[1]
libSQL Vector Query	5 - 15 ms	DiskANN index with vector_top_k.[15]
Gemini 1.5 Flash TTFT	150 - 220 ms	Explicit Context Caching + Thinking Budget = 0.[28]
Stream Injection	< 10 ms	Non-blocking Deno TransformStream.[36]
Total Perception	**199 - 289 ms**	

The perception of latency is further reduced by the use of streaming Markdown parsers on the client side, which render interactive tooltips as soon as a new token is received, rather than re-parsing the entire message.[1]

Conclusion

The architecture of the LocalPlus MVP represents a synthesis of 2026 edge-computing standards and generative intelligence optimizations. By deploying the Model Context Protocol on Bunny.net's Anycast-routed Magic Containers, the system ensures that compute is physically adjacent to users in Bangkok and Sydney, eliminating the latency inherent in centralized architectures. The use of libSQL's DiskANN vector search provides the necessary speed to query a live-graph of local merchants in under 50ms, while Gemini 1.5 Flash's context caching and thinking-budget controls allow for a generative TTFT that fits within the sub-300ms budget.

The integration of real-time Symbolic Injection via Deno Edge Scripting creates a transparent and interactive monetization layer, allowing for the transition from a link-based economy to an answer-based economy. Finally, the "Direct Push" capability ensures that the AI's grounding is

never more than 500ms behind the physical reality of the merchant's status. At an estimated operational cost of approximately $22 per month for the MVP scale, this technical roadmap provides a sustainable and scalable foundation for the next generation of local discovery.

Works cited

1. Research 5 _ Live-Graph Software Stack Research.md
2. What is Anycast and how does it work? - Bunny.net, accessed March 22, 2026, https://bunny.net/academy/network/what-is-anycast-and-how-does-it-work/
3. Global anycast in a click: How Magic Containers simplifies low-level global networking, accessed March 22, 2026, https://bunny.net/blog/global-anycast-in-a-click-how-magic-containers-simplifies-low-level-networking/
4. Magic Containers - bunny.net Documentation, accessed March 22, 2026, https://docs.bunny.net/magic-containers
5. Deploy - Developer Hub - bunny.net Documentation, accessed March 22, 2026, https://docs.bunny.net/magic-containers/deploy
6. Step-by-Step: Building a Docker-Based MCP from Scratch | by DEEPANSHU KHANNA | NeoScoop | Medium, accessed March 22, 2026, https://medium.com/neoscoop/step-by-step-building-a-docker-based-mcp-from-scratch-b409da9244d0
7. Endpoints - bunny.net Documentation, accessed March 22, 2026, https://docs.bunny.net/magic-containers/endpoints
8. How to expose your app to the internet - bunny.net Support Hub, accessed March 22, 2026, https://support.bunny.net/hc/en-us/articles/14391726266012-How-to-expose-your-app-to-the-internet
9. Steps to build MCP server Docker Image - Generative AI Application Builder on AWS, accessed March 22, 2026, https://docs.aws.amazon.com/solutions/latest/generative-ai-application-builder-on-aws/steps-to-build-mcp-server-docker-image.html
10. Docker — MCP Hello client-server using FastMCP library | by Ion - Medium, accessed March 22, 2026, https://medium.com/@ion.stefanache0/docker-mcp-hello-client-server-using-fastmcp-8d5b902c1af7
11. Creating an MCP (Model Context Protocol) Server on OpenShift | by ..., accessed March 22, 2026, https://medium.com/@shrishs/creating-an-mcp-model-context-protocol-server-on-openshift-85dceac65c07
12. Building a Server-Sent Events (SSE) MCP Server with FastAPI - Ragie, accessed March 22, 2026, https://www.ragie.ai/blog/building-a-server-sent-events-sse-mcp-server-with-fastapi
13. Bunny Database - bunny.net Documentation, accessed March 22, 2026, https://docs.bunny.net/database

14. Why we're building a developer toolbox to hop closer to your users - Bunny.net, accessed March 22, 2026, https://bunny.net/blog/why-were-building-a-developer-toolbox-to-hop-closer-to-your-users/
15. AI & Embeddings - Turso, accessed March 22, 2026, https://docs.turso.tech/features/ai-and-embeddings
16. SQL Database Vector Search - Azion Documentation, accessed March 22, 2026, https://www.azion.com/en/documentation/products/store/sql-database/vector-search/
17. Best Vector Databases in 2026: A Complete Comparison Guide - Firecrawl, accessed March 22, 2026, https://www.firecrawl.dev/blog/best-vector-databases
18. .NET - bunny.net Documentation, accessed March 22, 2026, https://docs.bunny.net/database/connect/dotnet
19. AI & Embeddings - Turso, accessed March 22, 2026, https://docs.turso.tech/features/ai-and-embeddings#vector-top-k
20. Native Vector Search for SQLite - Turso, accessed March 22, 2026, https://turso.tech/vector
21. vector_top_k JOIN with regular table causes pathological query plan (3700x slower than subquery) · Issue #2212 · tursodatabase/libsql - GitHub, accessed March 22, 2026, https://github.com/tursodatabase/libsql/issues/2212
22. How to evaluate the performance of a vector database? - Tencent Cloud, accessed March 22, 2026, https://www.tencentcloud.com/techpedia/138543
23. Evaluating Vector Search Quality: A Practical Guide for Developers - Shift Asia, accessed March 22, 2026, https://shiftasia.com/community/evaluating-vector-search-quality/
24. Gemini 1.5 Flash - Learn Prompting, accessed March 22, 2026, https://learnprompting.org/docs/models/gemini-1.5-flash
25. System prompt handling in Gemini - Google Help, accessed March 22, 2026, https://support.google.com/gemini/thread/340196124/system-prompt-handling-in-gemini?hl=en
26. High Latency and Rate Limit concerns for a New AI Startup – Seeking advice on Tier upgrading and Latency optimization - Gemini API - Google AI Developers Forum, accessed March 22, 2026, https://discuss.ai.google.dev/t/high-latency-and-rate-limit-concerns-for-a-new-ai-startup-seeking-advice-on-tier-upgrading-and-latency-optimization/114442
27. Context caching | Gemini API | Google AI for Developers, accessed March 22, 2026, https://ai.google.dev/gemini-api/docs/caching
28. [Guide] Stop wasting $ on Gemini tokens: 5 Engineering Tips for 1.5/2.0/3.0 : r/Bard, accessed March 22, 2026, https://www.reddit.com/r/Bard/comments/1q8w98s/guide_stop_wasting_on_gemini_tokens_5_engineering/
29. Response time for Gemini API - Gemini Apps Community - Google Help, accessed March 22, 2026, https://support.google.com/gemini/thread/312465300/response-time-for-gemini-api?hl=en

30. How can I reduce Gemini 2.5 Flash Lite latency to <400ms? : r/googlecloud - Reddit, accessed March 22, 2026, https://www.reddit.com/r/googlecloud/comments/1lwevh0/how_can_i_reduce_gemini_25_flash_lite_latency_to/
31. Context caching overview | Generative AI on Vertex AI - Google Cloud Documentation, accessed March 22, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/context-cache/context-cache-overview
32. Gemini thinking | Gemini API - Google AI for Developers, accessed March 22, 2026, https://ai.google.dev/gemini-api/docs/thinking
33. Long context | Generative AI on Vertex AI - Google Cloud Documentation, accessed March 22, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/long-context
34. Use system instructions | Generative AI on Vertex AI | Google Cloud Documentation, accessed March 22, 2026, https://docs.cloud.google.com/vertex-ai/generative-ai/docs/learn/prompts/system-instructions
35. Safety settings | Gemini API | Google AI for Developers, accessed March 22, 2026, https://ai.google.dev/gemini-api/docs/safety-settings
36. How to Handle Streams in Deno - OneUptime, accessed March 22, 2026, https://oneuptime.com/blog/post/2026-01-31-deno-streams/view
37. A beginner's guide to streams in Deno | Tech Tonic - Medium, accessed March 22, 2026, https://medium.com/deno-the-complete-reference/a-beginners-guide-to-streams-in-deno-760d51750763
38. TransformStream() constructor - Web APIs - MDN Web Docs, accessed March 22, 2026, https://developer.mozilla.org/en-US/docs/Web/API/TransformStream/TransformStream
39. SQL API - bunny.net Documentation, accessed March 22, 2026, https://docs.bunny.net/database/connect/sql-api
40. Bunny Database: The SQL service that just works, accessed March 22, 2026, https://bunny.net/database/
41. Edge Scripting | Make the edge your cloud. - Bunny.net, accessed March 22, 2026, https://bunny.net/edge-scripting/
42. Web Streams at the Edge - Deno, accessed March 22, 2026, https://deno.com/blog/deploy-streams
43. Best practices to render streamed LLM responses | AI on Chrome, accessed March 22, 2026, https://developer.chrome.com/docs/ai/render-llm-responses